Tupã: Uma Linguagem de Domínio Específico em Português para Programação de Microcontroladores ESP32

Michel Lutegar
Engenharia de Computação
IBMEC
Rio de Janeiro, Brasil
email@aluno.ibmec.edu.br

André Costa
Engenharia de Computação
IBMEC
Rio de Janeiro, Brasil
email@aluno.ibmec.edu.br

Felipe Dutrain
Engenharia de Computação
IBMEC
Rio de Janeiro, Brasil
202402011359@alunos.ibmec.edu.br

Guilherme Lopes
Engenharia de Computação
IBMEC
Rio de Janeiro, Brasil
202401632473@aluno.ibmec.edu.br

***Resumo*—Programar microcontroladores ainda exige domínio de C++, de bibliotecas específicas e da configuração manual de pinos, o que representa uma barreira significativa para iniciantes, estudantes e entusiastas de eletrônica. Este artigo apresenta o Tupã, uma linguagem de domínio específico com sintaxe em português e eventos de hardware como construções de primeira classe, voltada para a placa ESP32. O compilador, implementado em Python, percorre as quatro fases clássicas de compilação, análise léxica, análise sintática, análise semântica e geração de código, e produz um programa Arduino em C++ destinado à compilação e à gravação na placa pela Arduino CLI, etapa que ainda é manual. A análise semântica verifica o uso correto dos módulos de hardware antes da gravação e emite mensagens de erro didáticas em português. Em um exemplo de controle de acesso por cartão NFC, um programa de 12 linhas de código em Tupã gera 100 linhas de código C++, cerca de 8,3 vezes mais, sem que o usuário precise escrever inclusões de bibliotecas, definições de pinos ou as funções de configuração e laço principal. Os programas gerados compilam com a Arduino CLI, e um teste de bancada com firmware escrito à mão validou o protótipo com ESP32-WROVER, leitor RC522 e sensor infravermelho de obstáculo. O projeto encontra-se em desenvolvimento, com as quatro fases do compilador implementadas e a gravação do código gerado na placa ainda pendente.**

***Index Terms*—compiladores, linguagem de domínio específico, sistemas embarcados, ESP32, Arduino, ensino de programação**

I. Introdução

A popularização de placas de baixo custo, como Arduino e ESP32, aproximou estudantes e entusiastas do desenvolvimento de sistemas embarcados. Ainda assim, o caminho entre a ideia e o hardware funcionando costuma ser longo: mesmo uma aplicação simples, como acender um LED quando um cartão válido é aproximado de um leitor, exige incluir bibliotecas, definir pinos, inicializar periféricos e organizar o código nas funções `setup()` e `loop()` do ambiente Arduino. Para quem não domina C++, a configuração do hardware frequentemente consome mais tempo do que a lógica da aplicação.

Somam-se a isso a barreira do idioma, já que palavras-chave, bibliotecas e documentação estão majoritariamente em inglês, e o fato de que ferramentas educacionais existentes ou dependem de hardware fechado, ou trabalham com blocos visuais que não evoluem para uma linguagem textual. Há ainda uma barreira de ordem prática: erros de configuração, como um pino trocado ou um método inexistente, só aparecem na compilação do C++ ou, pior, em tempo de execução, com mensagens em inglês e pouco ligadas à intenção de quem escreveu o programa.

Este trabalho propõe o *Tupã*, uma linguagem de domínio específico (DSL, do inglês *domain-specific language*) com sintaxe em português, cujo compilador traduz programas de alto nível para C++ compatível com o ecossistema Arduino e executável no ESP32. A premissa central é que o usuário descreva *o que* o dispositivo deve fazer, e não *como* o hardware deve ser configurado, delegando ao compilador o trabalho de baixo nível. O projeto foi desenvolvido na disciplina de Linguagens Formais e Compiladores do IBMEC e cobre o ciclo completo: da especificação da linguagem à execução de um protótipo físico, passando por um ambiente web para experimentação.

A. Objetivos

O *objetivo geral* é projetar e implementar uma DSL em português para programar o ESP32, com um compilador que percorra as fases clássicas e gere código Arduino válido para a placa-alvo. Os *objetivos específicos* são:

1) definir a especificação léxica e a gramática da linguagem, com eventos de hardware como construções de primeira classe;

2) implementar as fases de análise léxica, sintática e semântica e a geração de código em C++/Arduino;
3) catalogar os erros do compilador com códigos estáveis e mensagens didáticas em português;
4) oferecer um ambiente web público para experimentar a linguagem sem instalar nada;
5) montar um protótipo de hardware (ESP32, leitor NFC, sensor infravermelho, botão e LEDs) e validar a geração de código contra ele;
6) medir a economia de código e o custo em memória em relação ao C++ equivalente.

B. Contribuições

As principais contribuições são:

- o projeto de uma linguagem textual em português com eventos de hardware de primeira classe, como `ao botao(1) { ... }`;
- um compilador completo, implementado em Python, que percorre as fases de análise léxica, sintática e semântica e de geração de código;
- uma análise semântica orientada ao hardware, que detecta erros antes da gravação na placa e produz mensagens didáticas em português;
- uma avaliação preliminar que compara o código escrito em Tupã com o código C++ equivalente gerado.

O restante do artigo está organizado da seguinte forma: a Seção II apresenta a fundamentação teórica e os trabalhos relacionados; a Seção III descreve a metodologia; a Seção IV define a linguagem; a Seção V detalha cada fase do compilador; a Seção VI trata do ambiente web; a Seção VII descreve o hardware; a Seção VIII apresenta os resultados; a Seção IX discute limitações; e a Seção X conclui e aponta trabalhos futuros.

II. Fundamentação Teórica e Trabalhos Relacionados

A. Linguagens de Domínio Específico

Uma DSL é uma linguagem de expressividade restrita, voltada a um domínio de problema particular, em contraste com as linguagens de propósito geral [2]. Mernik, Heering e Sloane discutem quando vale a pena criar uma DSL e como desenvolvê-la, e destacam que o ganho vem de notações próximas do vocabulário do domínio, ao custo de projetar e manter a linguagem e suas ferramentas [2]. Fowler distingue DSLs *internas*, embutidas em uma linguagem hospedeira e que aproveitam sua sintaxe e seu compilador, de DSLs *externas*, com sintaxe própria e analisador próprio [3]. A abordagem de DSLs embutidas em uma linguagem hospedeira também é discutida por Hudak [4]. O Tupã é uma DSL externa: tem gramática própria, o que permite palavras-chave em português e mensagens de erro sob medida, mas exige a construção de todas as fases do compilador.

B. Fases de um Compilador

A organização clássica de um compilador [1] separa um *front-end*, que analisa o programa-fonte, de um *back-end*, que sintetiza o programa de saída. O front-end compreende a análise léxica, que agrupa caracteres em *tokens* por meio de expressões regulares; a análise sintática, que verifica a estrutura segundo uma gramática livre de contexto e produz uma árvore sintática; e a análise semântica, que confere nomes, tipos e demais restrições que a gramática não expressa. O back-end gera o código de saída. No Tupã, essas quatro fases são módulos independentes de um pipeline linear.

C. Transpilação para C++ e Plataforma ESP32/Arduino

Um transpilador (ou compilador fonte a fonte) traduz um programa para outra linguagem de alto nível, em vez de código de máquina. O Tupã adota essa estratégia: gera um *sketch* Arduino em C++, e a cadeia de ferramentas existente (Arduino CLI [7] e o núcleo `esp32` da Espressif) cuida da compilação para o binário. A vantagem é reaproveitar bibliotecas maduras, como a MFRC522 [13], e o gerenciamento de placas. O ESP32 [12] é um microcontrolador com Wi-Fi e Bluetooth, programável pelo ambiente Arduino, em que o programa se organiza nas funções `setup()`, executada uma vez, e `loop()`, executada repetidamente [6].

D. Trabalhos Relacionados

O *Arduino IDE* [6] consolidou-se como a principal porta de entrada para a programação de microcontroladores. Sua linguagem, um subconjunto de C++ acompanhado de bibliotecas, é poderosa e madura, mas exige do usuário conhecimento de C++, do mapeamento de pinos e das APIs de cada periférico.

O *Scratch* [5] mostrou que uma linguagem de blocos, com foco em baixa barreira de entrada, atrai milhões de jovens para a programação. O *Microsoft MakeCode* [8] e o *Blockly* [9] estendem a ideia a microcontroladores e a ambientes educacionais. O MakeCode, em particular, compila programas de blocos ou JavaScript para código nativo em placas educacionais. Embora essas ferramentas reduzam a barreira inicial, não geram código Arduino legível nem oferecem ao estudante uma linguagem textual em português para a qual ele possa migrar.

O *Flipper Zero* [10] inspira o conceito de um dispositivo multifuncional para NFC, RFID e infravermelho, porém trata-se de hardware fechado e de custo elevado, sem foco em linguagem ou em ensino.

No contexto brasileiro, o *Portugol Studio* [11] mostrou o valor pedagógico de uma linguagem com palavras-chave em português para o ensino de algoritmos, mas não é voltado a sistemas embarcados.

O Tupã ocupa a lacuna entre essas abordagens: uma linguagem textual em português, que compila para C++/Arduino nativo e executa no ESP32, pensada para *makers* e para o ensino de sistemas embarcados.

III. Metodologia e Desenvolvimento

A. Processo do Grupo e Cronograma

O grupo, de quatro estudantes, trabalhou de forma incremental: primeiro a especificação da linguagem (tokens, gramática e exemplo canônico), depois o compilador fase a fase,

na ordem do *pipeline*, e por fim o ambiente web e a integração com o hardware. Cada fase só foi considerada pronta com testes automatizados. O cronograma do semestre, organizado em torno das entregas da disciplina, foi o seguinte: pitch em setembro (plano de negócios, pesquisa, marca, orçamento e documentação no Git); vídeo tutorial e artigo em outubro; documentação completa no Git, cobrindo léxico, sintático, semântico, geração de código e tabela de erros, e demonstração na placa em novembro. O teste de bancada do hardware ocorreu em 26/10/2026.

B. Ferramentas

O compilador é escrito em Python 3, sem dependências além da biblioteca padrão. Os testes usam *pytest*. O ambiente web usa FastAPI, React com CodeMirror e Docker (Seção VI). A compilação do C++ gerado e a medição de memória usam a Arduino CLI 1.5.1, o núcleo `esp32:esp32` 3.3.12 e a biblioteca MFRC522 1.4.12. O artigo é escrito em LaTeX com a classe IEEEtran.

A interface de linha de comando, `tupa`, expõe uma fase por subcomando: `tupa tokens` imprime a análise léxica, `tupa ast` imprime a árvore, `tupa simbolos` imprime a tabela de símbolos e `tupa build arquivo.tupa -o pasta -placa p` gera o `.ino`. Essa separação foi útil para depurar e para documentar cada fase com saídas reais. O compilador encerra com status 1 em erros de linguagem e 2 em erros de placa ou de arquivo.

C. Testes

Há três grupos de testes automatizados, hoje em 98 casos: 41 do compilador (`test_compilador.py`), 46 do catálogo de erros (`test_erros.py`) e 11 da API web (`test_web_api.py`). Os testes do catálogo executam o exemplo mínimo de cada erro e exigem o código e a mensagem esperados, conferem que a documentação gerada está sincronizada com o catálogo e que todo código citado no fonte existe no catálogo. Além disso, o código gerado é compilado de fato com a Arduino CLI.

D. Uso de Git

O código, a documentação e o artigo ficam em um repositório Git público, com licença MIT, em https://github.com/mlutegar/compilador-project. O histórico registra cada etapa do desenvolvimento em *commits*, atualmente 23. A documentação em `docs/` traz a especificação léxica, a tabela de tokens, a análise sintática e semântica, a tabela de erros e a evidência de compilação para a placa.

IV. A Linguagem Tupã

A. Princípios de Projeto

O Tupã foi projetado segundo três princípios: (i) *sintaxe em português*, com palavras-chave como `se`, `senao`, `inteiro` e `verdadeiro`; (ii) *eventos de hardware de primeira classe*, em que reações a estímulos físicos são construções da própria linguagem, e não chamadas de biblioteca; e (iii) *abstração de periféricos por módulos*, como `nfc`, `led` e `ir`, cujos detalhes de pinagem e inicialização ficam a cargo do compilador.

B. Exemplo Canônico

A Listagem 1 apresenta um controle de acesso por cartão: ao pressionar o botão 1, o dispositivo lê um cartão NFC e acende o LED verde se o cartão for válido, ou o vermelho caso contrário.

```
dispositivo controle {
    inteiro tentativas = 3;

    ao botao(1) {
        # le um cartao NFC
        cartao = nfc.ler();

        se cartao.valido == verdadeiro {
            led.verde();
            esperar(1500);
        } senao {
            led.vermelho();
        }
    }
}
```

Listagem 1. Controle de acesso por cartão NFC em Tupã (`controle_nfc.tupa`).

Um programa Tupã é composto por um bloco `dispositivo`, que pode declarar variáveis de estado e um ou mais tratadores de evento introduzidos pela palavra-chave `ao`. Dentro de um tratador, o usuário utiliza estruturas de controle e chamadas a métodos dos módulos de hardware. Os eventos disponíveis são `ao iniciar`, `ao botao(N)`, `ao obstaculo` e `ao cartao`. Os tipos de dado são `inteiro`, `decimal`, `texto` e `booleano`.

V. Arquitetura do Compilador

O compilador do Tupã é implementado em Python 3 e segue a arquitetura clássica de compiladores [1], organizada em quatro fases encadeadas, ilustradas na Fig. 1. A saída é um arquivo `.ino`, que deve então ser compilado e gravado no ESP32 pela Arduino CLI [7]; essa etapa não é executada pelo compilador.

A. Análise Léxica

O analisador léxico converte o texto-fonte em uma sequência de *tokens*, registrando linha e coluna de cada um para uso nas mensagens de erro das fases seguintes. O vocabulário é composto por 44 tipos de *token*: 43 enviados ao analisador sintático mais o marcador sintético de fim de entrada (`TK_EOF`). Espaços e comentários de linha, iniciados por `#`, são reconhecidos e descartados, e por isso não entram na contagem. A Tabela I lista os 44 tipos.

Os nomes de evento `iniciar`, `obstaculo` e `cartao` não são *tokens* próprios: chegam como identificadores e são conferidos na análise semântica, o que mantém `cartao` livre como nome de variável. Do mesmo modo, métodos como `ler` e `verde` e propriedades como `valido` são identificadores comuns, validados pela análise semântica.

A técnica é a de expressões regulares: uma única expressão mestre reúne todas as alternativas como grupos nomeados, na forma `(?P<TK_ESPACO>...)|(?P<TK_COMENTARIO>...)|(?P<TK_FLOAT>...)`

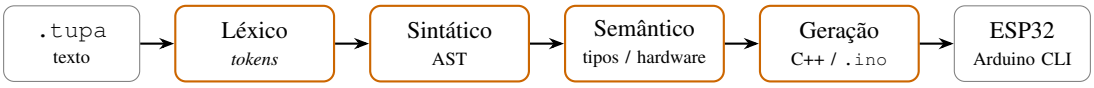


Figura 1. Fluxo de compilação do Tupã, do código-fonte à gravação na placa.

Tabela I
OS 44 TIPOS DE *token* DA LINGUAGEM TUPÃ (PREFIXO TK_ OMITIDO)

Token	Lexema / padrão	Token	Lexema / padrão	Token	Lexema / padrão
DISPOSITIVO	dispositivo	NFC	nfc	IGUAL	==
AO	ao	IR	ir	DIFERENTE	!=
SE	se	LED	led	MAIOR	>
SENAO	senao	BOTAO	botao	MENOR	<
REPETIR	repetir	ESPERAR	esperar	MAIOR_IGUAL	>=
INTEIRO	inteiro	ESCREVER	escrever	MENOR_IGUAL	<=
DECIMAL	decimal	ID	[a-zA-Z][a-zA-Z0-9_]*	SOMA	+
TEXTO	texto	INT	[0-9]+	SUB	-
BOOLEANO	booleano	FLOAT	[0-9]+\.[0-9]+([eE][+-]?[0-9]+)?	MULT	*
VERDADEIRO	verdadeiro	STRING	"[^"\n]*"	DIV	/
FALSO	falso	ATRIB	=	E	&&
RETORNAR	retornar	ABRE_PAR	(	OU	||
VIRGULA	,	FECHA_PAR	)	NAO	!
PONTO	.	ABRE_CHAVE	{	PONTO_VIRGULA	;
EOF	fim da entrada	FECHA_CHAVE	}		

`|...`, e é aplicada de forma iterativa sobre o texto, o que dá a cada *token* o grupo que casou primeiro. A ordem das alternativas é significativa: espaços e comentários vêm primeiro, os literais decimais precedem os inteiros, e os operadores de dois caracteres, como == e >=, precedem os de um caractere. As palavras reservadas não têm expressão própria: o identificador casado é reclassificado por consulta a um dicionário. Um caractere que nenhuma alternativa reconhece produz o erro léxico L01, com linha e coluna.

Como exemplo real, a Tabela II mostra a tokenização da linha 8 do exemplo canônico, `cartao = nfc.ler();`, obtida com o comando `tupa tokens`. Note que `ler` é um `TK_ID`, e não uma palavra reservada.

Tabela II
TOKENIZAÇÃO DE `cartao = nfc.ler();` (LINHA 8 DE `controle_nfc.tupa`)

Token	Lexema	Linha:coluna
TK_ID	cartao	8:9
TK_ATRIB	=	8:16
TK_NFC	nfc	8:18
TK_PONTO	.	8:21
TK_ID	ler	8:22
TK_ABRE_PAR	(	8:25
TK_FECHA_PAR	)	8:26
TK_PONTO_VIRGULA	;	8:27

B. Análise Sintática

O analisador sintático consome os *tokens* e constrói uma árvore sintática abstrata (AST) que representa o dispositivo, suas declarações e seus tratadores de evento. A técnica é a de descendência recursiva preditiva, com um *token* de antecipação e sem retrocesso: cada não-terminal da Fig. 2 é um método do analisador. A Fig. 2 apresenta a gramática completa, em notação EBNF, que o analisador efetivamente aceita. Além de `ao botao(N)`, há os eventos `ao iniciar` (uma vez, na partida), `ao obstaculo` (mudança do sensor infravermelho) e `ao cartao` (cartão novo lido, com `cartao.uid` e `cartao.valido`), e o comando `escrever`, que envia um valor ao Monitor Serial. As regras de expressão estão agrupadas por nível de precedência, da menor para a maior, obtida por uma cadeia de métodos, um por nível, com laços que produzem associatividade à esquerda. O primeiro erro sintático interrompe a compilação, sem recuperação.

```
programa  ::= "dispositivo" ID "{" item* "}" EOF
item      ::= evento | comando
evento    ::= "ao" "botao" "(" INT ")" bloco
            | "ao" nome_evento bloco
nome_evento ::= "iniciar" | "obstaculo" | "cartao"
bloco     ::= "{" comando* "}"
comando   ::= decl_var | se | repetir | esperar
            | escrever | retornar | atrib_ou_chamada
decl_var  ::= tipo ID ("=" expr)? ";"
tipo      ::= "inteiro" | "decimal" | "texto" | "booleano"
se        ::= "se" expr bloco ("senao" bloco)?
repetir   ::= "repetir" "(" expr ")" bloco
esperar   ::= "esperar" "(" expr ")" ";"
escrever  ::= "escrever" "(" expr ")" ";"
retornar  ::= "retornar" expr? ";"
atrib_ou_chamada ::= expr ("=" expr)? ";"
expr      ::= e ("||" e)*
e         ::= igualdade ("&&" igualdade)*
igualdade ::= comparacao (("==" | "!=") comparacao)*
comparacao ::= termo ((">" | "<" | ">=" | "<=") termo)*
termo     ::= fator (("+" | "-") fator)*
fator     ::= unaria (("*" | "/") unaria)*
unaria    ::= ("!" | "-") unaria | posfixo
posfixo   ::= primario ("." ID | "(" args? ")")*
args      ::= expr ("," expr)*
primario  ::= INT | FLOAT | STRING | "verdadeiro" | "falso"
            | "nfc" | "ir" | "led" | "botao" | ID
            | "(" expr ")"
```

Figura 2. Gramática do Tupã em EBNF, conforme o analisador sintático.

A AST possui 17 tipos de nó, todos com linha e coluna: `Programa`, `DeclVar`, `Evento`, `Atribuicao`, `Se`, `Repetir`, `Esperar`, `Escrever`, `Retornar`,

`ExprComando`, `Binaria`, `Unaria`, `Literal`, `Identificador`, `Hardware`, `AcessoMembro` e `Chamada`. A Fig. 3 mostra a AST do exemplo canônico, usando 11 desses tipos.

C. Análise Semântica

A análise semântica é o que diferencia o Tupã de um simples tradutor de texto. Além das verificações usuais, como tipos e declaração de variáveis, ela valida o uso dos módulos de hardware: cada módulo expõe um conjunto conhecido de métodos, e chamadas inexistentes são rejeitadas antes de qualquer código ser gerado. A fase mantém uma *pilha de escopos* (dicionários de nome para tipo), aberta em cada evento, `se`, `senao` e `repetir`, e uma *tabela de símbolos* permanente, com nome, tipo, escopo, linha e coluna de cada variável, impressa pelo comando `tupa simbolos`. Para o exemplo canônico, a tabela contém `tentativas` (`inteiro`, escopo global, 3:5) e `cartao` (tipo `cartao`, escopo `ao botao(1)`, 8:16). A variável `cartao` é criada pela primeira atribuição, o que permite escrever `cartao = nfc.ler();` sem declaração.

As verificações cobrem: existência e unicidade de nomes; compatibilidade de tipos (`decimal` aceita `inteiro`, os demais só o próprio tipo); condições booleanas em `se` e `!`; operandos numéricos em `esperar` e `repetir`; existência de módulo, método e propriedade e número de argumentos; validade e unicidade dos eventos; e dependências da placa, pois `ao botao(N)` exige um pino para o botão N e `ao obstaculo` exige o pino do sensor IR. Cada erro tem um código estável, em quatro famílias: **L** (léxico), **S** (sintático), **E** (semântico) e **G** (geração e arquivo de placa). A Tabela III resume o catálogo, que no total tem 1 erro léxico, 13 sintáticos, 21 semânticos (dois deles defesas internas) e 5 de geração.

Tabela III
RESUMO DA TABELA DE ERROS (CATÁLOGO COMPLETO EM `DOCS/TABELA-DE-ERROS.MD`)

Código	Fase	Condição
L01	léxico	caractere que nenhum *token* reconhece
S01 a S13	sintático	`dispositivo`, nome, chaves, parênteses, número do botão, nome de variável, `;`, nome de membro, nome de evento, expressão ou código após o fim ausentes ou fora de lugar
E01 a E05	semântico	evento: sem pino IR, desconhecido, botão inválido, botão sem pino, repetido
E07, E08	semântico	`escrever` de chamada sem valor ou de objeto inteiro
E09, E12	semântico	variável redeclarada ou não declarada
E10, E21	semântico	alvo de atribuição inválido; tipos incompatíveis
E11, E14, E20	semântico	condição, `!` ou `esperar`/`repetir` com tipo errado
E13, E16 a E19	semântico	módulo sem método, propriedade ou método inexistente, argumentos errados
E06, E15	interno	defesas contra nó desconhecido
G01 a G05	placa/arquivo	placa inexistente, UID inválido, campo ausente, arquivo inexistente

Os módulos de hardware e seus métodos, que a análise consulta em um dicionário único, são os da Tabela IV. O tipo `cartao` tem as propriedades `valido` (booleano) e `uid` (texto). A checagem do número de argumentos e do tipo de retorno vem dessa mesma tabela, e acrescentar um método à linguagem é acrescentar uma entrada nela e a tradução correspondente no gerador.

Tabela IV
API DE HARDWARE CONHECIDA PELA ANÁLISE SEMÂNTICA

Módulo	Método	Retorno
`led`	`verde()`, `vermelho()`, `apagar()`	vazio
`nfc`	`ler()`	`cartao`
`ir`	`detectado()`	booleano
`botao`	`pressionado()`	booleano

A Fig. 4 mostra a mensagem real emitida quando o programador chama um método inexistente no módulo `led` (E19): o erro é localizado por linha e coluna, explicado em português e acompanhado da lista de métodos disponíveis, em tom didático. A Fig. 5 mostra o erro E04, que nasce da dependência entre programa e placa: o botão 2 não tem GPIO no arquivo da placa, e a mensagem diz como declarar um.

D. Geração de Código

O gerador percorre a AST validada e emite um programa Arduino completo: as inclusões de bibliotecas necessárias, como a MFRC522 para o leitor NFC [13]; as definições de pinos; a inicialização dos periféricos em `setup()`; e a tradução dos tratadores de evento para verificações dentro de `loop()`. Bibliotecas e pinos só são emitidos quando o programa usa o módulo correspondente.

A pinagem não está fixa no gerador: vem de um *arquivo de placa* em JSON (`compilador/placas/esp32_wrover.json`, escolhido pela opção `-placa`), que traz o FQBN da Arduino CLI, a pinagem, o modo elétrico dos sensores e a lista de UIDs autorizados. A Listagem 2 mostra um trecho. Trocar de placa passa a ser trocar de arquivo.

```
{ "nome": "esp32_wrover",
  "fqbn": "esp32:esp32:esp32wrover",
  "pinos": { "nfc_ss": 5, "nfc_rst": 22,
             "led_verde": 25, "led_vermelho": 26,
             "ir": 34, "botoes": { "1": 4 } },
  "eletrica": {
    "botao": { "modo": "INPUT_PULLUP", "ativo_em": "LOW" },
    "ir":    { "modo": "INPUT", "ativo_em": "LOW" } },
  "nfc": { "timeout_ms": 3000,
           "uids_autorizados": ["33 7A 87 F7",
                                "43 A3 6F F5"] } }
```

Listagem 2. Trecho do arquivo de placa `esp32_wrover.json`.

O mapeamento dos eventos para C++ é resumido na Tabela V. Os eventos de botão e de obstáculo detectam *bordas*, e não níveis, de modo que cada ação dispara uma única vez.

A Listagem 3 traz o trecho de `loop()` gerado a partir do exemplo canônico, com a ligação entre blocos e linhas do fonte em comentários.

```
void loop() {
  // [controle_nfc.tupa:5] ao botao(1)
  {
    int leitura = digitalRead(BOTAO_1);
```

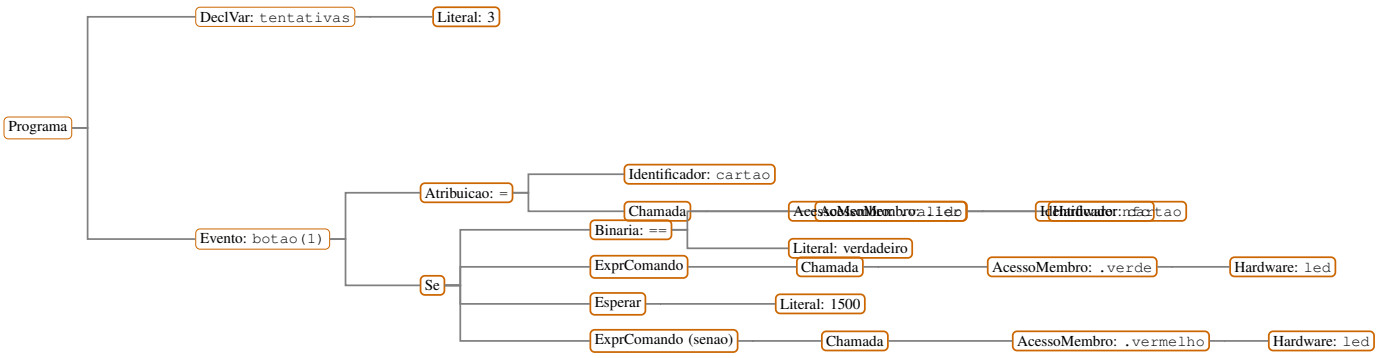


Figura 3. AST de `controle_nfc.tupa`. Os filhos de `Se` são, em ordem, a condição, o bloco `então` (dois comandos) e o bloco `senao`.

```
erro semantico [E19] - linha 3, coluna 12
    metodo inexistente no modulo 'led'
    disponiveis: verde(), vermelho(), apagar()
    led.roxo()

compilacao interrompida: 1 erro
```

Figura 4. Erro semântico E19, saída real do compilador.

```
erro semantico [E04] - linha 2, coluna 5
    a placa 'esp32_wrover' não tem pino configurado
    para o botao 2
    botoes com pino: 1
    para usar o botao 2, ligue-o a um GPIO e declare
    em "pinos.botoes" de compilador/placas/esp32_wrover.json
    ao botao(2)
```

Figura 5. Erro semântico E04, saída real do compilador.

```
    if (leitura != botao1_leitura) {
      botao1_leitura = leitura; botao1_mudou = millis(); }
    if ((millis() - botao1_mudou) > DEBOUNCE_MS
        && leitura != botao1_estavel) {
      botao1_estavel = leitura;
      if (botao1_estavel == LOW) {
        Cartao cartao = nfcLer();
        if ((cartao.valido == true)) {
          ledVerde();
          delay(1500);
        } else {
          ledVermelho();
        }
        ledApagar();
      }
    }
  }
  delay(10);
}
```

Listagem 3. Trecho do `controle_nfc.ino` gerado.

Três decisões de projeto merecem destaque. O *debounce* usa `millis()`, sem `delay()`, com janela de 50 ms e ação na borda. A chamada `nfc.ler()` espera até 3 s por um cartão e, sem cartão, devolve um resultado inválido; ao fim da leitura o cartão é encerrado com `PICC_HaltA` e `PCD_StopCrypto1`. A validação compara o UID lido com a lista de UIDs autorizados da placa, convertida em uma tabela constante em C++. Em `setup()`, o código verifica o registrador de versão do RC522 e avisa pelo Serial se o leitor não responde. O compilador grava o resultado em `<pasta>/<pasta>.ino`, formato exigido pela Arduino CLI.

Tabela V
MAPEAMENTO DE EVENTOS E COMANDOS TUPÃ PARA C++

Tupã	**C++ gerado**
`ao iniciar`	corpo acrescentado ao final de `setup()`
`ao botao(N)`	bloco em `loop()` com leitura de `digitalRead`, *debounce* por `millis()` e ação na borda de descida
`ao obstaculo`	bloco em `loop()` que compara a leitura do pino IR com a anterior; ação quando vai a LOW
`ao cartao`	`if (nfcTentar(cartao))` em `loop()`, sem espera
`nfc.ler()`	`nfcLer()`: espera até 3 s e devolve `Cartao`
`led.verde()`	`ledVerde()`: `digitalWrite` nos dois LEDs
`esperar(ms)`	`delay(ms)`
`escrever(x)`	`Serial.println(x)`
`se/senao/repetir`	`if/else/for`

VI. Ambiente Web

O projeto inclui um *playground* que expõe o compilador no navegador, publicado em https://tupa.mlutegar.com. Para o código digitado, ele mostra a lista de *tokens*, a AST, os erros, com código, fase, linha e coluna, e o C++ gerado, além de métricas de linhas (Fig. 6).

A arquitetura tem dois componentes. O serviço é um aplicativo FastAPI [14], que reutiliza o pacote do compilador sem duplicar lógica e expõe, entre outros, o ponto `/api/compilar` e a consulta ao catálogo de erros. O *front-end* é um aplicativo React [15] com editor CodeMirror, construído com Vite e servido pelo próprio serviço. O empacotamento usa Docker [16], em duas etapas: a primeira constrói o *front-end*, e a segunda, sobre uma imagem Python enxuta, executa o serviço com usuário sem privilégios e *healthcheck*. O *deploy* roda esse contêiner atrás do domínio público. Como o compilador, o ambiente apenas gera o `.ino`. O repositório é público e o código está sob licença MIT [17].

VII. Plataforma de Hardware

O protótipo utiliza uma placa ESP32-WROVER-DEV [12], um leitor NFC/RFID RC522, controlado pela biblioteca MFRC522 [13], um sensor infravermelho de obstáculo, dois

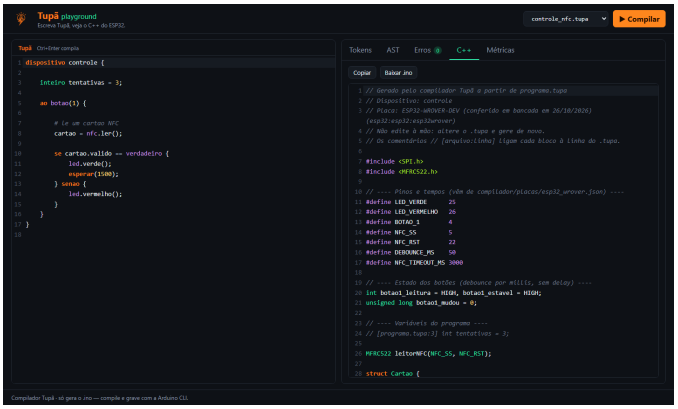


Figura 6. Ambiente web do Tupã em `tupa.mlutegar.com`: editor, *tokens*, AST, erros e C++ gerado.

LEDs e um botão. A pinagem, conferida em bancada, está na Tabela VI, e é a mesma do arquivo de placa. O sensor infravermelho é um módulo de obstáculo, com emissor e receptor juntos e saída digital, e não um controle remoto; o GPIO 34 é apenas de entrada e não tem *pull-up* interno.

Tabela VI
PINAGEM DA ESP32-WROVER-DEV (CONFERIDA EM BANCADA EM 26/10/2026)

Função	GPIO	Modo
RC522 SS	5	SPI
RC522 RST	22	saída
RC522 SCK / MISO / MOSI	18 / 19 / 23	SPI, 3,3 V
Botão 1	4	`INPUT_PULLUP`, ativo em LOW
LED verde	25	`OUTPUT`
LED vermelho	26	`OUTPUT`
Sensor IR de obstáculo	34	`INPUT`, ativo em LOW

A Fig. 7 mostra a montagem em detalhe. A Tabela VII lista os materiais com custo estimado de varejo brasileiro por protótipo, cerca de R$ 120 no total. Os valores são estimativas, e não preços de nota fiscal. Todo o software utilizado é de código aberto, sem custo de licença.

Tabela VII
LISTA DE MATERIAIS E CUSTO ESTIMADO (R$)

Item	Custo
ESP32 DevKit (WROVER)	40
Leitor NFC/RFID RC522	15
Sensor IR de obstáculo	10
LEDs e resistores	5
Botões	5
Protoboard e *jumpers*	30
Cabo USB	15
Total	**120**

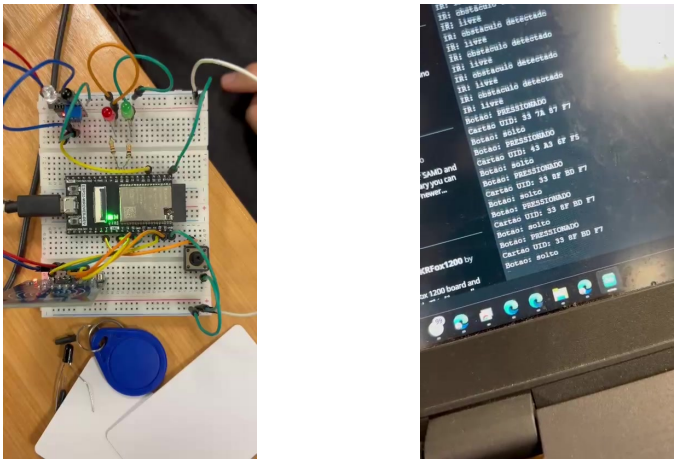


Figura 7. Vista superior da montagem (esquerda) e saída do Monitor Serial com os três UIDs lidos (direita).

VIII. RESULTADOS

As quatro fases do compilador estão implementadas, e um conjunto de 98 testes automatizados, 41 do compilador, 46 do catálogo de erros e 11 da API do ambiente web, é executado com sucesso. Dois programas são traduzidos de ponta a ponta até um arquivo `.ino`: o exemplo canônico da Listagem 1 (`controle_nfc.tupa`) e o teste de hardware em Tupã (`teste_hardware.tupa`), que usa os eventos novos e `escrever`. Os dois arquivos gerados e os dois firmwares escritos à mão compilam sem erro nem aviso com a Arduino CLI para a placa ESP32 Wrover Module (`esp32:esp32:esp32wrover`).

O teste de bancada, de 26/10/2026, foi feito com o firmware escrito à mão (`firmware/teste_hardware.ino`), e a Fig. 8 mostra a montagem e o Monitor Serial. O teste leu três UIDs distintos (`33 7A 87 F7`, `43 A3 6F F5` e `33 8F BD F7`) e registrou o botão e o sensor infravermelho. As imagens, portanto, são do firmware manual: o código gerado pelo compilador ainda *não* foi gravado nem executado na placa.

O segundo programa, `teste_hardware.tupa` (Listagem 4), reproduz em Tupã o firmware manual de bancada e exercita os quatro eventos. Suas 23 linhas de código geram um *sketch* que acende cada LED por 700 ms na partida e registra no Serial cada aperto do botão, cada mudança do sensor IR e o UID de cada cartão lido. O evento `ao obstaculo` não precisa de corpo, porque o *log* sai sozinho.

```
dispositivo teste_hardware {
    ao iniciar {
        escrever("== Teste de hardware Tupa ==");
        led.verde();    esperar(700);
        led.vermelho(); esperar(700);
    }
    ao botao(1) { led.vermelho(); esperar(300); }
    ao obstaculo { }
    ao cartao {
        escrever("Cartao UID: " + cartao.uid);
        led.verde();  esperar(800);
    }
}
```

Listagem 4. Resumo de `teste_hardware.tupa`.

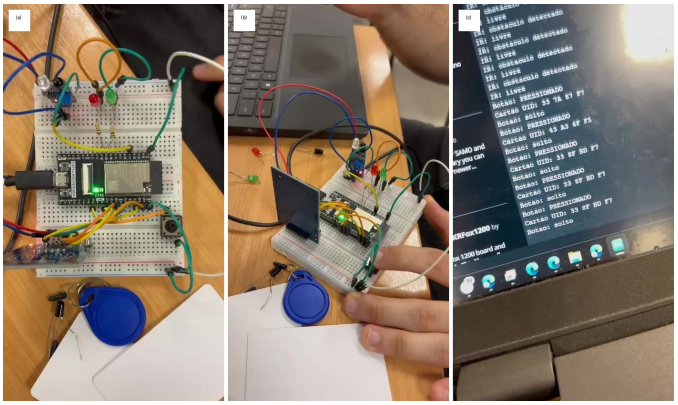


Figura 8. Protótipo em funcionamento: (a) montagem em protoboard com ESP32-WROVER, leitor RC522, sensor IR de obstáculo, botão e LEDs; (b) leitura de uma tag RFID; (c) Monitor Serial registrando eventos do sensor IR, do botão e os UIDs lidos.

A Tabela VIII compara os programas em Tupã com o C++ gerado e, para o teste de hardware, com o C++ que seria escrito à mão (`teste_hardware.ino`, a referência). O controle de acesso passou de 12 para 100 linhas de código, cerca de 8,3 vezes mais, e o teste de hardware de 23 para 112, cerca de 4,9 vezes. O gerado tem cerca de 2,2 vezes as linhas do manual (112 contra 51), porque inclui *debounce* por `millis()` com detecção de borda, espera de 3 s no NFC, encerramento do cartão, conferência da lista de UIDs e funções auxiliares de LED, que o firmware manual não tem. Nenhuma linha de configuração do hardware precisa ser escrita pelo usuário.

Tabela VIII
TUPÃ VERSUS C++ GERADO E C++ MANUAL

	controle_nfc		teste_hardware		
Métrica	**Tupã**	**Gerado**	**Tupã**	**Gerado**	**Manual**
Linhas totais	17	126	33	139	67
Linhas de código	12	100	23	112	51
Inclusões de bibliotecas	0	2	0	2	2
Constantes (`#define`)	0	7	0	7	6
Razão C++/Tupã		8,3		4,9	

Linhas de código excluem linhas vazias e comentários. As duas inclusões são `SPI.h` e `MFRC522.h`. Nos arquivos gerados, as sete constantes são os pinos (cinco no `controle_nfc`, seis no `teste_hardware`), a janela de *debounce* e, no `controle_nfc`, o *timeout* do NFC. No manual, as seis constantes são pinos. Há uma diferença conhecida de comportamento: o firmware manual mantém o LED vermelho aceso enquanto o botão está apertado, e o programa em Tupã o acende por 300 ms a cada aperto.

A Tabela IX traz o uso de memória medido pela Arduino CLI. O código gerado custa cerca de 2 KB de flash e 24 bytes de RAM a mais que o manual equivalente, uma diferença desprezível diante dos cerca de 290 KB do núcleo do ESP32.

A Tabela X resume as métricas de verificação. Os campos marcados com TODO dependem da gravação do `.ino` gerado na placa.

Tabela IX
USO DE MEMÓRIA NA ESP32 WROVER MODULE (ARDUINO CLI 1.5.1, NÚCLEO 3.3.12)

Sketch	**Flash (B)**	**RAM (B)**
`controle_nfc` gerado	293 740 (22%)	22 308 (6%)
`teste_hardware` gerado	293 880 (22%)	22 308 (6%)
`controle_nfc` manual	291 776 (22%)	22 284 (6%)
`teste_hardware` manual	292 548 (22%)	22 300 (6%)

Tabela X
MÉTRICAS DE VERIFICAÇÃO DO PROJETO

Métrica	**Valor**
Tipos de *token*	44
Tipos de nó da AST	17
Códigos de erro catalogados	40
Testes do compilador	41
Testes do catálogo de erros	46
Testes da API web	11
Sketches compilados sem erro	4 de 4
Teste de bancada com o exttt.ino gerado	% TODO (placa)
Taxa de acerto no teste de bancada com o gerado	TODO (placa) %

IX. DISCUSSÃO E LIMITAÇÕES

Os resultados indicam que a DSL cumpre o papel de esconder a configuração do hardware: o usuário escreve de 12 a 23 linhas, e o compilador devolve de 100 a 112 linhas de C++ com *debounce*, *timeout*, encerramento do cartão e verificação do leitor, itens em que iniciantes costumam errar. O custo em memória é pequeno, e o catálogo de erros move para o tempo de compilação falhas que, em C++, só apareceriam na placa.

As limitações são as seguintes. (i) Só há suporte à ESP32-WROVER-DEV; outras placas exigem um novo arquivo JSON e, possivelmente, ajustes no gerador. (ii) O compilador reporta um erro por vez e não se recupera de erros sintáticos. (iii) O módulo `ir` é só de sensor de obstáculo, sem transmissão de infravermelho nem controle remoto. (iv) Não há atualização do firmware por Wi-Fi (OTA); a gravação é manual, pela Arduino CLI. (v) A linguagem não foi avaliada com usuários, de modo que a hipótese de ganho didático ainda não foi testada. (vi) O código gerado compila, mas ainda não foi gravado na placa, portanto seu comportamento no hardware não foi verificado. Por fim, as métricas de linhas dependem do estilo de formatação do gerador e comparam com um único firmware manual de referência, o que limita a generalização.

Quanto à validade dos resultados, a redução de linhas mede esforço de digitação, e não esforço cognitivo: ela não prova que o Tupã seja mais fácil de aprender, apenas que ele dispensa código repetitivo. A comparação com C++ usa um firmware manual escrito pelos próprios autores, que poderiam tê-lo escrito de modo mais enxuto ou mais robusto. Além disso, o teste de bancada validou o hardware e a pinagem, e não o código gerado. Por isso, os números de memória e de linhas

devem ser lidos como evidência de viabilidade, e não como avaliação definitiva.

X. Conclusão e Trabalhos Futuros

Este artigo apresentou o Tupã, uma linguagem de domínio específico em português para programação de microcontroladores ESP32, e seu compilador, que percorre as quatro fases clássicas de compilação e gera C++ compatível com o ecossistema Arduino. Os resultados indicam que a linguagem reduz significativamente a quantidade de código que o usuário precisa escrever e elimina a configuração manual do hardware, enquanto a análise semântica orientada a módulos detecta erros antes da gravação. O protótipo com ESP32-WROVER, RC522, sensor infravermelho de obstáculo, botão e LEDs foi validado em bancada com um firmware escrito à mão, e os programas gerados compilam com a Arduino CLI para a placa-alvo. Falta, porém, gravar o `.ino` gerado na placa e repetir o teste de bancada com ele, de modo que o funcionamento do código gerado no hardware ainda não foi verificado.

Os próximos passos incluem: gravar na placa o código gerado por `controle_nfc.tupa` e `teste_hardware.tupa` e comparar seu comportamento com o do firmware manual; automatizar a compilação e a gravação a partir do comando `tupa build`; ampliar os módulos suportados, como o envio de infravermelho, hoje fora do escopo; suportar outras placas além da ESP32-WROVER; permitir a atualização do firmware por Wi-Fi (OTA); e avaliar o uso da linguagem com estudantes em contexto de ensino de sistemas embarcados.

Agradecimentos

Os autores agradecem ao professor da disciplina de Linguagens Formais e Compiladores do IBMEC pela orientação.

Referências

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley, 2006.

[2] M. Mernik, J. Heering, and A. M. Sloane, "When and how to develop domain-specific languages," *ACM Comput. Surv.*, vol. 37, no. 4, pp. 316–344, Dec. 2005.

[3] M. Fowler, *Domain-Specific Languages*. Upper Saddle River, NJ: Addison-Wesley, 2010.

[4] P. Hudak, "Building domain-specific embedded languages," *ACM Comput. Surv.*, vol. 28, no. 4es, art. 196, Dec. 1996.

[5] M. Resnick *et al.*, "Scratch: programming for all," *Commun. ACM*, vol. 52, no. 11, pp. 60–67, Nov. 2009.

[6] Arduino, "Arduino language reference." [Online]. Available: https://docs.arduino.cc/language-reference/. Acesso em: nov. 2026.

[7] Arduino, "Arduino CLI." [Online]. Available: https://arduino.github.io/arduino-cli/. Acesso em: nov. 2026.

[8] J. Devine, J. Finney, P. de Halleux, M. Moskal, T. Ball, and S. Hodges, "MakeCode and CODAL: intuitive and efficient embedded systems programming for education," *J. Syst. Archit.*, vol. 98, pp. 468–483, 2019.

[9] N. Fraser, "Ten things we've learned from Blockly," in *Proc. IEEE Blocks and Beyond Workshop*, 2015, pp. 49–50.

[10] Flipper Devices, "Flipper Zero." [Online]. Available: https://flipperzero.one/. Acesso em: nov. 2026.

[11] A. Esteves, L. Noschang, A. Raabe, and A. Filho, "Portugol Studio: em direção a uma comunidade aberta para pesquisa sobre o aprendizado de programação," in *Anais do Workshop sobre Educação em Computação (WEI/CSBC)*, 2019, pp. 513–522.

[12] Espressif Systems, "ESP32 series datasheet." [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents. Acesso em: nov. 2026.

[13] NXP Semiconductors, "MFRC522: standard performance MIFARE and NTAG frontend," product data sheet.

[14] S. Ramírez, "FastAPI." [Online]. Available: https://fastapi.tiangolo.com/. Acesso em: nov. 2026.

[15] Meta Open Source, "React." [Online]. Available: https://react.dev/. Acesso em: nov. 2026.

[16] Docker, Inc., "Docker." [Online]. Available: https://www.docker.com/. Acesso em: nov. 2026.

[17] Open Source Initiative, "The MIT License." [Online]. Available: https://opensource.org/license/mit. Acesso em: nov. 2026.